Atividade Avaliativa: Banco de Dados & Programação OO

Mapeamento Objeto-Relacional e Implementação de Sistema de Gestão Escolar (DDL e DML)

Parte 1: Analogia entre Tabelas do Banco de Dados e Classes em Java

A partir da análise do material ministrado na **Aula03.pdf** (que aborda a estrutura do banco de dados relacional de e-commerce composto pelas tabelas `clientes`, `pedidos`, `produtos/livros` e `itens_pedidos`), realizou-se o mapeamento das tabelas relacionais para classes do paradigma de Orientação a Objetos em Java.

Quantidade de Classes: 4

Em Java, aplicamos a convenção **PascalCase** e nomes no **singular**. Foram mapeadas **4 classes** correspondentes às entidades do banco de dados:

1. Classe: `Cliente` (Representa a tabela `clientes`)

- `int idCliente` (ou `Long`) — Chave Primária (PK)
- `String nome` — Nome do cliente
- `String email` — E-mail de contato (único)
- `String cpf` — Cadastro de Pessoa Física (único)
- `String telefone` — Telefone de contato
- `LocalDateTime dataCadastro` — Data/hora de criação do registro

2. Classe: `Produto` (Representa a tabela `produtos` / `livros`)

- `int idProduto` (ou `Long`) — Chave Primária (PK)
- `String nome` (ou `titulo`) — Descrição/Título do produto
- `String descricao` — Detalhes do item
- `double preco` (ou `BigDecimal`) — Valor monetário atual
- `int estoque` — Quantidade disponível
- `LocalDateTime dataCadastro` — Data de inclusão no catálogo

3. Classe: `Pedido` (Representa a tabela `pedidos`)

- `int idPedido` (ou `Long`) — Chave Primária (PK)
- `Cliente cliente` — Referência à classe `Cliente` (na modelagem OO, a FK `id_cliente` torna-se uma associação direta para o objeto Cliente)
- `LocalDateTime dataPedido` — Data/hora da realização da compra
- `String status` — Situação do pedido (ex: 'Pendente', 'Concluído')
- `double total` (ou `BigDecimal`) — Valor total do pedido
- `List<ItemPedido> itens` — Lista com os itens pertencentes ao pedido (Relacionamento 1:N)

4. Classe: `ItemPedido` (Representa a tabela associativa `itens_pedidos`)

- `int idItem` (ou `Long`) — Identificador único da linha
- `Pedido pedido` — Objeto referente ao pedido pai
- `Produto produto` — Objeto referente ao produto selecionado
- `int quantidade` — Quantidade do item no pedido
- `double precoUnitario` (ou `BigDecimal`) — Preço unitário praticado na venda

Parte 2: Sistema de Gestão Escolar (DDL e DML)

Para o cenário do **Sistema de Gestão Escolar**, foram modeladas 4 tabelas relacionais com cardinalidades apropriadas e integridade referencial. Abaixo encontram-se os scripts DDL de criação e DML de população com dados fictícios.

1. Script DDL — Criação do Banco e Tabelas

```sql
-- ============================================================
-- SCRIPT DDL: CRIAÇÃO DO BANCO DE DADOS E ESTRUTURA DAS TABELAS
-- ============================================================

CREATE DATABASE IF NOT EXISTS gestao_escolar;
USE gestao_escolar;

-- 1. Tabela de Alunos
CREATE TABLE IF NOT EXISTS alunos (
    id_aluno INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(11) NOT NULL UNIQUE,
    data_nascimento DATE NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    telefone VARCHAR(15)
);

-- 2. Tabela de Professores
CREATE TABLE IF NOT EXISTS professores (
    id_professor INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cpf VARCHAR(11) NOT NULL UNIQUE,
    especialidade VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    data_admissao DATE NOT NULL
);

-- 3. Tabela de Disciplinas
CREATE TABLE IF NOT EXISTS disciplinas (
    id_disciplina INT AUTO_INCREMENT PRIMARY KEY,
    nome_disciplina VARCHAR(100) NOT NULL,
    carga_horaria INT NOT NULL,
    id_professor INT,
    FOREIGN KEY (id_professor) REFERENCES professores(id_professor) ON DELETE SET NULL
);

-- 4. Tabela de Matrículas (Relacionamento Aluno-Disciplina)
CREATE TABLE IF NOT EXISTS matriculas (
```

```
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,
    id_aluno INT NOT NULL,
    id_disciplina INT NOT NULL,
    data_matricula DATE NOT NULL,
    nota_final DECIMAL(4, 2),
    status VARCHAR(20) DEFAULT 'Ativo',
    FOREIGN KEY (id_aluno) REFERENCES alunos(id_aluno) ON DELETE CASCADE,
    FOREIGN KEY (id_disciplina) REFERENCES disciplinas(id_disciplina) ON DELETE CASCADE
);
```

2. Script DML — Inserção de Dados Fictícios

```
-- ============================================================
-- SCRIPT DML: INSERÇÃO DE DADOS (MÍNIMO 3 REGISTROS POR TABELA)
-- ============================================================

USE gestao_escolar;

-- Inserindo Professores
INSERT INTO professores (nome, cpf, especialidade, email, data_admissao) VALUES
('Carlos Eduardo', '11122233344', 'Banco de Dados', 'carlos.eduardo@escola.com',
'2020-02-15'),
('Mariana Luz', '22233344455', 'Programação Java', 'mariana.luz@escola.com', '2019-08-01'),
('Roberto Almeida', '33344455566', 'Engenharia de Software', 'roberto.almeida@escola.com',
'2021-03-10');

-- Inserindo Alunos
INSERT INTO alunos (nome, cpf, data_nascimento, email, telefone) VALUES
('Ana Clara Souza', '44455566677', '2002-05-14', 'ana.souza@aluno.com', '11988887777'),
('Lucas Gabriel', '55566677788', '2001-11-20', 'lucas.gabriel@aluno.com', '11977776666'),
('Beatriz Lima', '66677788899', '2003-01-30', 'beatriz.lima@aluno.com', '11966665555');

-- Inserindo Disciplinas
INSERT INTO disciplinas (nome_disciplina, carga_horaria, id_professor) VALUES
('Modelagem de Banco de Dados', 80, 1),
('Programação Orientada a Objetos em Java', 100, 2),
('Arquitetura de Sistemas', 60, 3);

-- Inserindo Matrículas
INSERT INTO matriculas (id_aluno, id_disciplina, data_matricula, nota_final, status) VALUES
(1, 1, '2026-02-01', 9.50, 'Aprovado'),
(2, 2, '2026-02-01', 8.00, 'Cursando'),
(3, 1, '2026-02-02', 7.50, 'Cursando'),
(1, 3, '2026-02-03', 8.80, 'Aprovado');
```